

환불 요청 중계 서버

쇼핑몰의 환불 요청을 받아, 외부 결제대행사에 안전하게 전달하는 서버를 만듭니다.

💡 이 문서는 요약본입니다. 처음 과제를 시작하시는 분들을 위해 큰 그림만 먼저 보여드려요. 자세한 API 규격, 시나리오, 힌트는 저장소의 README.md 에 전부 정리되어 있으니, 이 문서를 다 보신 뒤 README를 차근차근 따라가시면 됩니다. 급하게 외우지 않아도 괜찮습니다 — 만들면서 다시 찾아보면 됩니다.

1개

구현할 서버

환불 요청 중계 서버 하나만 새로 만들면 됩니다

2가지

꼭 지켜야 할 약속

여러분 서버의 API 형식 + 결제대행사과 주고받는 방식

13개

점검 시나리오

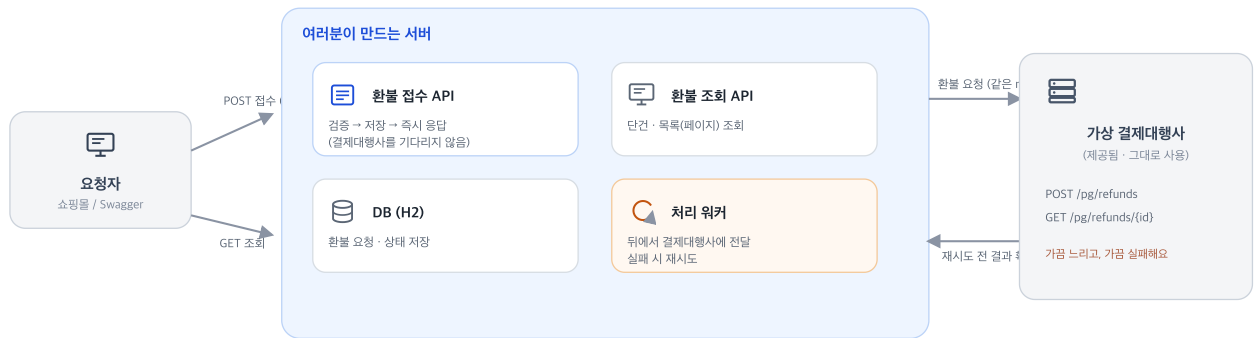
이 시나리오가 모두 통과하면 과제 완성입니다

제한 없음

소요 시간

빠르게보다 정확하게 만드는 것이 목표예요

전체 구조 — 요청이 어떻게 흘러가나요



접수(파란색)는 결제대행사를 기다리지 않고 즉시 응답하고, 처리 워커(주황색)가 뒤에서 따로 전달합니다 — 이 과제의 핵심 설계 포인트예요.

이 과제에서 이루려는 것 / 신경 쓰지 않아도 되는 것



목표 Outcome

- 환불 요청을 빠짐없이 접수하고 **바로 응답**합니다
- 결제대행사에 전달을 **보장**합니다 (실패해도 재시도)
- 같은 요청은 **절대 두 번 처리하지 않습니다**
- 환불 한도, 상태, 이력을 정확히 관리합니다



신경 쓰지 않아도 되는 것 Not in scope

- 실제 결제·환불 연동 (가상 결제대행사로 대체)
- 로그인, 인증/인가 (이 과제 범위 밖)
- Kafka 등 메시지 브로커 도입 (필요하지 않아요)
- 대용량 트래픽 성능 튜닝 (기능 정확성이 우선)

제공되는 것 — 가상 결제대행사

여러분이 직접 만들 필요는 없습니다. Docker로 바로 띄워서 사용하는 **모의(mock) 서버**입니다. 다만 아래 세 가지 특징을 **일부러** 가지고 있어요 — 실제 외부 시스템이 흔히 그렇듯이요.

가끔 느려요

응답이 몇 초씩 걸릴 수 있어요. 접수 API가 이걸 기다리면 안 됩니다.

가끔 실패해요

일부러 실패·오류 응답을 보냅니다. 재시도 로직을 검증하기 위해서예요.

같은 ID는 기억해요

같은 요청 ID로 다시 물어보면 "이미 처리됨"을 알려줍니다. 이중 처리를 막는 열쇠예요.

가장 중요한 함정 — "느린 건데, 사실은 이미 처리됨"

결제대행사 응답을 기다리다 **시간 초과로 실패**라고 판단했는데, 사실 결제대행사는 이미 환불을 끝낸 경우가 있습니다. 이때 무작정 재시도하면 **환불이 두 번** 일어납니다. 아래 흐름을 기억해 두세요.

① 요청

워커가 결제대행사에 같은 refundId로 환불을 요청합니다



② 오해

응답이 너무 늦어 "실패"로 보고 재시도를 준비합니다 (사실은 성공 처리됨)



③ 확인

같은 ID로 재시도하면 결제대행사가 "이미 처리됨"이라고 알려줍니다



④ 방지

새로 환불을 만들지 않고 그 결과를 그대로 반영합니다

자세한 시퀀스와 API 응답 형식은 README.md 6장에 정리되어 있어요.

약속 요약 — 이것만은 형식을 지켜주세요

항목	규칙
환불 접수	POST /api/v1/refunds — 정상 접수는 202, 같은 ID 중복은 200, 한도 초과는 422
환불 조회	GET /api/v1/refunds, GET /api/v1/refunds/{refundId} — 최신순, 페이지 처리
상태	ACCEPTED → COMPLETED / FAILED, 거절 시 REJECTED — 이 4가지로만 외부에 보여줍니다
재시도	최초 시도 포함 최대 4번 시도, 같은 refundId 로만 재시도
결제대행사	POST /pg/refunds, GET /pg/refunds/{refundId} — 이미 제공되어 있고 그대로 사용

점검 시나리오 — 이 13개가 통과하면 완성입니다

① 접수 단계 (S1~S7)

7개

- 정상 접수는 바로 202
- 같은 ID·같은 내용 재요청 → 200 (중복 처리)
- 같은 ID·다른 내용 → 409
- 같은 ID로 동시에 여러 건 → 정확히 1건만 저장
- 한도 초과 → 422, REJECTED로 기록
- 같은 주문에 여러 건 동시 요청 → 한도 안에서만 접수
- 필수값 누락 → 400

② 처리 단계 (S8~S11)

4개

- 결제대행사 성공 → COMPLETED
- 실패·오류 → 재시도 후에도 안 되면 FAILED
- 응답 시간 초과 → 실패로 보고 재시도
- 같은 주문은 한 건씩 순서대로 처리

③ 조회 (S13)

1개

- 주문별 목록 조회 — 최신순 · 페이지 처리

④ 심화 · 선택 (S12, 가점)

1개

- 응답이 늦었지만 실제로는 처리된 경우에도 환불이 딱 1번만 일어남

처음이라면, 이렇게 시작해 보세요

1 이 문서로 큰 그림을 먼저 파악하세요.

어떤 서버를 왜 만드는지, 무엇이 이미 제공되는지 확인했다면 절반은 준비된 거예요.

2 README.md를 처음부터 끝까지 한 번 읽어보세요.

정확한 API 형식, 에러 코드, 13개 시나리오가 전부 담겨 있어요. 외우지 말고 훑어만 보세요.

3 `docker compose up` 으로 가상 결제대행사부터 띄워보세요.

내 서버보다 먼저 실행해서 Swagger로 어떻게 동작하는지 눈으로 확인하세요.

4 `starter-project` 를 열고 작은 것부터 만드세요.

정상 접수(S1) 하나가 되는 걸 먼저 확인한 뒤, 시나리오를 하나씩 늘려가세요.

5 막히면 README 11장의 힌트를 하나씩만 열어보세요.

정답을 알려주진 않지만, 어느 방향으로 생각해야 할지 짚어줍니다.

🙌 정답이 하나로 정해진 과제가 아닙니다. 왜 그렇게 설계했는지 설명할 수 있다면, 그것으로 충분해요.

자세한 내용은 저장소의 README.md를 확인해 주세요. 화이팅입니다!